



Machine-Level Programming II: Condition Code and Conditional Branch

CMSC 14400

Yanjing Li



Today

- **Control: Condition codes**

Consider this code..

we do this by
condition code

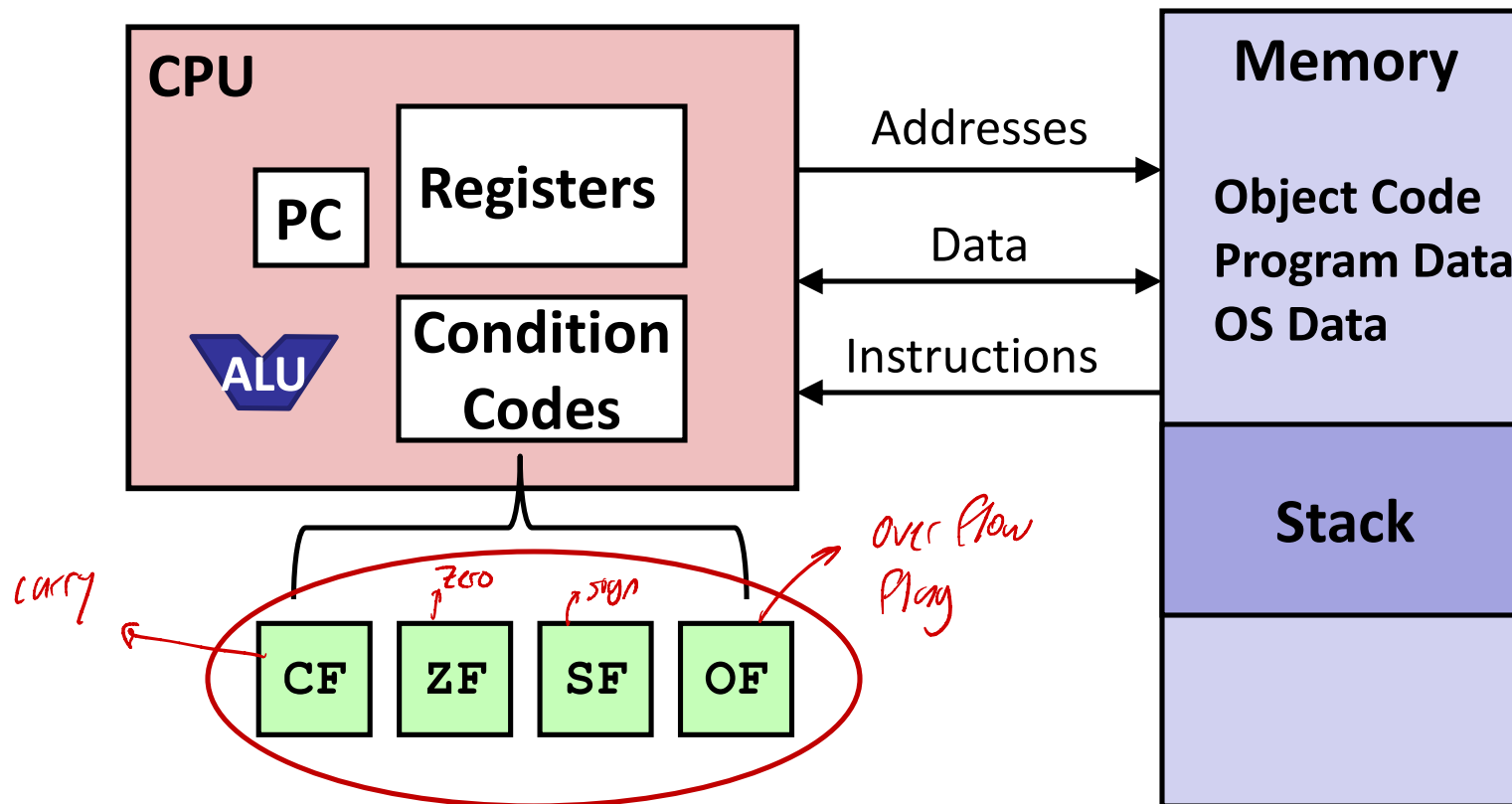
```
long less (long x, long y)
{
    return x < y;
}
```

```
long max (long x, long y)
{
    if (x < y) return x;
    else return y;
}
```

Need something to keep track of the "**side effect**" of an arithmetic operation

In these examples, we need to know
whether $x - y < 0$ or not

Control Flow in x86

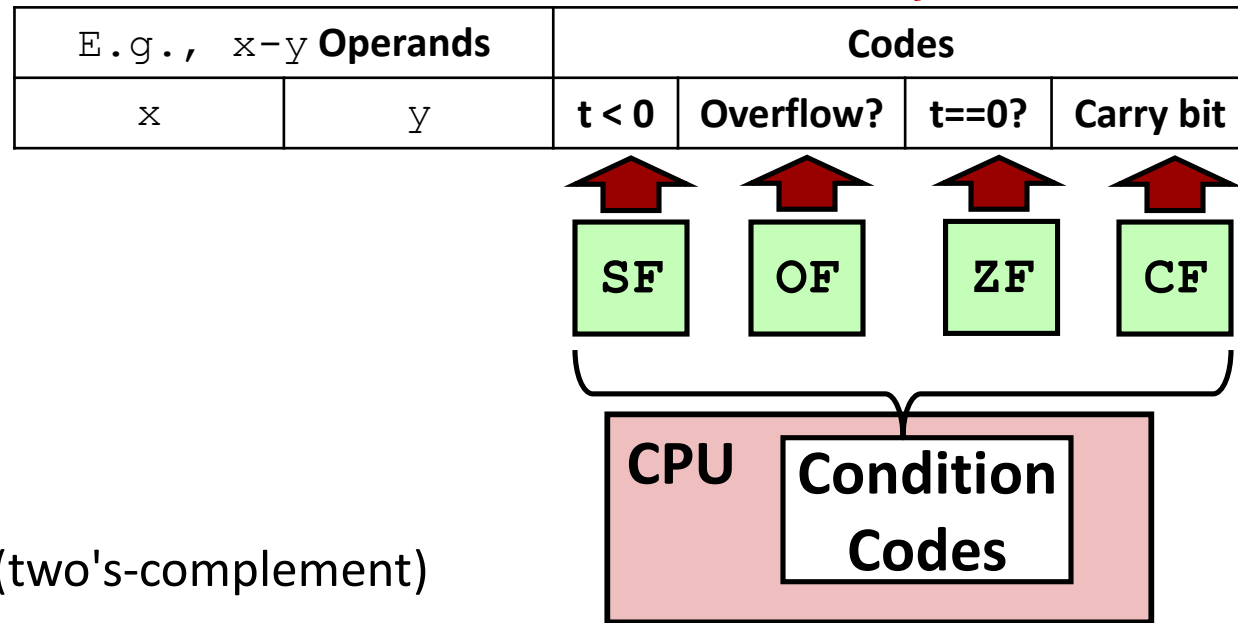


Condition codes record different types of side effects



x86 condition codes

*not in memory, these
are next to register*



SF Sign flag = 1, if $t < 0$ (two's-complement)

OF Overflow flag = 1, if two's-complement
(signed) overflow *↳ signed overflow*

ZF Zero flag = 1, if $t == 0$

CF Carry flag = 1, if carry out from most
significant bit (unsigned overflow) *if unsigned,
carry flag set to 1 if
there's an overflow*

Example: Consider $x < y$ (signed)

Compute $t = x - y$
 $x < y$ iff t is negative



Is this sufficient?

Consider $x < y$ (signed)

- Do a signed (two's complement) $x - y$ and save the result in t
 - Assume 8-bit machine

Operands		Result	
x	y	$t = x + (-y)$	$t < 0?$
00001100 (12)	00000011 (3)	$\begin{array}{r} 00001100 \text{ (12)} \\ + 11111101 \text{ (-3)} \\ \hline 1\text{ }00001001 \text{ (9)} \end{array}$	No

Consider $x < y$ (signed)

- Do a signed (two's complement) $x - y$ and save the result in t
 - Assume 8-bit machine

Operands		Result	
x	y	$t = x + (-y)$	$t < 0?$
00001100 (12)	00000011 (3)	00001001 (9)	No
00000011 (3)	00001100 (12)	00000011 (3) +11110100 (-12) 11110111 (-9)	Yes

Consider $x < y$ (signed)

- Do a signed (two's complement) $x - y$ and save the result in t
 - Assume 8-bit machine

Operands		Result	
x	y	$t = x + (-y)$	$t < 0?$
00001100 (12)	00000011 (3)	00001001 (9)	No
00000011 (3)	00001100 (12)	11110111 (-9)	Yes
11001011 (-53)	01001110 (78)	$\begin{array}{r} 11001011 \text{ (-53)} \\ + 10110010 \text{ (-78)} \\ \hline 01111101 \text{ (125)} \end{array}$	No

Negative
overflow

look at that
overflow

exceeds what we
can represent in 8 bits

Consider $x < y$ (signed)

- Do a signed (two's complement) $x - y$ and save the result in t
 - Assume 8-bit machine

Operands		Result	
x	y	$t = x + (-y)$	$t < 0?$
00001100 (12)	00000011 (3)	00001001 (9)	No
00000011 (3)	00001100 (12)	11110111 (-9)	Yes
11001011 (-53)	01001110 (78)	01111101 (125)	No
00110101 (53)	10110010 (-78)	00110101 (53) +01001110 (78) 10000011 (-125)	Yes

Negative
overflow

Positive
overflow

Positive over flow

Need another code to track overflow

Consider $x < y$ (signed)

■ Do a signed (two's complement) $x - y$ and save the result in t

- Assume 8-bit machine

the overflows are represented by these flags

Operands		Result	SF	OF
x	y	$t = x + (-y)$	$t < 0$	Overflow?
00001100 (12)	00000011 (3)	00001001 (9)	No	No
00000011 (3)	00001100 (12)	11110111 (-9)	Yes	No
11001011 (-53)	01001110 (78)	01111101 (125)	No	Yes
00110101 (53)	10110010 (-78)	10000011 (-125)	Yes	Yes

Correct statement: $x < y$ iff $(t \text{ is negative}) \wedge (\text{overflow})$

Correct statement: $x < y$ iff $SF \wedge OF$



What about $x \leq y$?

- Do a signed (two's complement) $x - y$ and save the result in t
 - Assume 8-bit machine

Zero val

Operands		Result	SF	OF	ZF
x	y	$t = x + (-y)$	$t < 0$	Overflow ?	$t == 0?$
00001100 (12)	00000011 (3)	00001001 (9)	No	No	No
00000011 (3)	00001100 (12)	11110111 (-9)	Yes	No	No
11001011 (-53)	01001110 (78)	01111101 (125)	No	Yes	No
00110101 (53)	10110010 (-78)	10000011 (-125)	Yes	Yes	No
00110101 (53)	00110101 (53)	$ \begin{array}{r} 00110101 (53) \\ + 11001011 (-53) \\ \hline 100000000 (0) \end{array} $	No	No	Yes

$x \leq y$ iff $(t \text{ is negative}) \wedge (\text{overflow})$ or t is zero

$x \leq y$ iff $SF \wedge OF$ or ZF

For the equal condition



What about **unsigned** $x < y$?

■ Do an **unsigned** subtraction $x - y$ and save the result in t

■ Assume 8-bit machine

Operands		Result	SF	OF	ZF	CF
x	y	$t = x - y$	$t < 0$	Overflow?	$t == 0$	Carry bit
00001100 (12)	00000011 (3)	$\begin{array}{r} 00001100 \text{ (12)} \\ - 00000011 \text{ (3)} \\ \hline 00001001 \text{ (9)} \end{array}$	No	No	No	No
00000011 (3)	00001100 (12)	$\begin{array}{r} 00000011 \text{ (3)} \\ - 00001100 \text{ (12)} \\ \hline 11110111 \text{ (-)} \end{array}$	Yes	No	No	Yes

$x < y$ iff carry bit is set to 1 (unsigned overflow) *First*

$x < y$ iff CF (unsigned overflow) *unsigned*

$x \leq y$ iff carry bit is set to 1 or t is zero *we need to check the*

$x \leq y$ iff CF or ZF *carry flag instead of OF*

Condition codes are *implicitly* set after arithmetic instructions (not just subtraction)

`subq, addq, andq, salq, incq, ...`

A few examples of adds (do it after the class)

	a	b	SF t < 0	OF Overflow?	ZF t==0?	CF Carry bit
add	00001100	00000011	0	0	0	0
	01000011	01001100	1	1	0	0
	00111100	10110000	1	0	0	0
	10000000	10000000	0	1	1	1
	11111111	11111111	1	0	0	1

we will also set flags
for many kinds of operations

Note: Codes are not set by `leaq`

Condition Codes (Explicit Setting: Test)

■ Explicit Setting by Test instruction

- `testq Src2, Src1`

- `testq b, a` like computing `a&b` without setting destination (unlike `andq`)

- **ZF set** when `a&b == 0`

- **SF set** when `a&b < 0`

- OF and CF are set to 0

a	b	SF	OF	ZF	CF
00001100	00000011	0	0	1	0
01000011	01001100	0	0	0	0
11111111	11111111	1	0	0	0

maybe we need

to explicitly set the flag

bitwise operations

Condition Codes (Explicit Setting: Compare)

■ Explicit Setting by Compare Instruction

- `cmpq b, a` like computing $a - b$ without setting destination (unlike `subq`)
- **CF set** if carry out from most significant bit (used for unsigned comparisons)
- **ZF set** if $a == b$
- **SF set** if $(a - b) < 0$ (as signed)
- **OF set** if two's-complement (signed) overflow

```
(a>0 && b<0 && (a-b)<0) || (a<0 && b>0 && (a-b)>=0)
    positive overflow ||    negative overflow
```

Comparing is like subtracting

Reading Condition Codes

■ SetX Instructions,

- Set all bits to 1 in the lowest byte of a register
- Does not alter remaining 7 bytes

SetX	Condition	Description
setl	$(SF \wedge OF)$	Less (Signed)
setne	$\sim ZF$	Not Equal / Not Zero
sets	SF	Negative
setns	$\sim SF$	Nonnegative
setg	$\sim (SF \wedge OF) \ \& \ \sim ZF$	Greater (Signed)
setge	$\sim (SF \wedge OF)$	Greater or Equal (Signed)
setl	$(SF \wedge OF)$	Less (Signed)
setle	$(SF \wedge OF) \mid ZF$	Less or Equal (Signed)
seta	$\sim CF \ \& \ \sim ZF$	Above (unsigned)
setb	CF	Below (unsigned)

Reading Condition Codes

■ SetX Instructions

- Does not alter remaining 7 bytes

SetX	Condition	Description
setl	(SF^OF)	Less (Signed)

```
long less (long x, long y)
{
    return x < y;
}
```

```
cmpq    %rsi,%rdi    # Compare x:y
setl     %al          # Set when <
movzbl   %al,%eax     # Zero rest of %rax
ret
```

changes the condition codes
x-y

doesn't change the lower byte

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

Reading Condition Codes

■ SetX Instructions

- Does not alter remaining 7 bytes

SetX	Condition	Description
setl	(SF^OF)	Less (Signed)

```
long less (long x, long y)
{
    return x < y;
}
```

```
cmpq    %rsi,%rdi    # Compare x:y
setl     %al          # Set when <
movzbl  %al,%rax      # Zero rest of %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

$x = 2, y = 3$

SF = ? OF = ?



%rax

%al

Reading Condition Codes

■ SetX Instructions

- Does not alter remaining 7 bytes

SetX	Condition	Description
setl	(SF^OF)	Less (Signed)

```
long less (long x, long y)
{
    return x < y;
}
```

```
cmpq    %rsi,%rdi    # Compare x:y
setl     %al          # Set when <
movzbl  %al,%rax     # Zero rest of %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

$x = 2, y = 3$

SF = **1** OF = **0**

$$\begin{array}{r} 00...10 \\ -00...11 \\ \hline 11...11 \end{array}$$


%rax

%al

Reading Condition Codes

■ SetX Instructions

- Does not alter remaining 7 bytes

SetX	Condition	Description
setl	(SF^OF)	Less (Signed)

Refer to textbook 3.4 on p180 for references to lower bytes of x86-64 registers

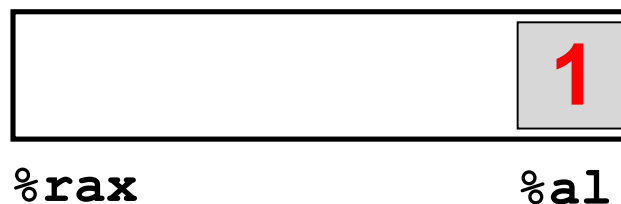
```
long less (long x, long y)
{
    return x < y;
}
```

```
cmpq    %rsi,%rdi    # Compare x:y
setl     %al          # Set when <
movzbq  %al,%rax     # Zero rest of %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

$x = 2, y = 3$

SF = **1** OF = **0**



Reading Condition Codes

■ SetX Instructions

- Does not alter remaining 7 bytes

SetX	Condition	Description
setl	(SF^OF)	Less (Signed)

Refer to textbook 3.4.2 on p184 for details on movzbl and similar instructions

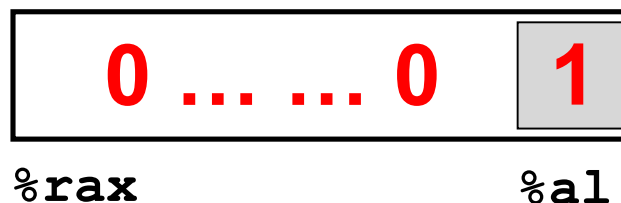
```
long less (long x, long y)
{
    return x < y;
}
```

```
cmpq    %rsi,%rdi    # Compare x:y
setl     %al          # Set when <
movzbl  %al,%rax     # Zero rest of %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

$x = 2, y = 3$

SF = **1** OF = **0**





Extra Materials

Arithmetic Expression Example

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```


Arithmetic Expression Example

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

arith:

```
movq    %rdx, %rax
leaq    (%rdi,%rsi), %rdx
addq    %rdx, %rax
leaq    (%rsi,%rsi,2), %rdx
salq    $4, %rdx
leaq    4(%rdi,%rdx), %rdx
imulq   %rdx, %rax
ret
```

Interesting Instructions

- **leaq**: address computation
- **salq**: left shift
- **imulq**: multiplication
 - But, only used once

Understanding Arithmetic Expression

Example

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

arith:

```
movq    %rdx, %rax
leaq    (%rdi,%rsi), %rdx
addq    %rdx, %rax
leaq    (%rsi,%rsi,2), %rdx
salq    $4, %rdx
leaq    4(%rdi,%rdx), %rdx
imulq   %rdx, %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z
%rax	??

Understanding Arithmetic Expression

Example

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

arith:

```
movq    %rdx, %rax
leaq     (%rdi,%rsi), %rdx
addq     %rdx, %rax
leaq     (%rsi,%rsi,2), %rdx
salq     $4, %rdx
leaq     4(%rdi,%rdx), %rdx
imulq    %rdx, %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z
%rax	z

Understanding Arithmetic Expression

Example

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

arith:

```
movq    %rdx, %rax
leaq    (%rdi,%rsi), %rdx
addq    %rdx, %rax
leaq    (%rsi,%rsi,2), %rdx
salq    $4, %rdx
leaq    4(%rdi,%rdx), %rdx
imulq   %rdx, %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	z → t1
%rax	z

Understanding Arithmetic Expression

Example

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

arith:

```
movq    %rdx, %rax
leaq    (%rdi,%rsi), %rdx
addq    %rdx, %rax
leaq    (%rsi,%rsi,2), %rdx
salq    $4, %rdx
leaq    4(%rdi,%rdx), %rdx
imulq   %rdx, %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	t1
%rax	z → t2

Understanding Arithmetic Expression

Example

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

arith:

```
movq    %rdx, %rax
leaq    (%rdi,%rsi), %rdx
addq    %rdx, %rax
leaq    (%rsi,%rsi,2), %rdx
salq    $4, %rdx
leaq    4(%rdi,%rdx), %rdx
imulq   %rdx, %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	t1 → t4
%rax	t2

Understanding Arithmetic Expression

Example

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

arith:

```
movq    %rdx, %rax
leaq    (%rdi,%rsi), %rdx
addq    %rdx, %rax
leaq    (%rsi,%rsi,2), %rdx
salq    $4, %rdx
leaq    4(%rdi,%rdx), %rdx
imulq   %rdx, %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	t4 → t5
%rax	t2

Understanding Arithmetic Expression

Example

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

arith:

```
movq    %rdx, %rax
leaq    (%rdi,%rsi), %rdx
addq    %rdx, %rax
leaq    (%rsi,%rsi,2), %rdx
salq    $4, %rdx
leaq    4(%rdi,%rdx), %rdx
imulq   %rdx, %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	t5
%rax	t2 → rval

rax holds the return value
(explained in future lectures)

Review: Signed integers with two's complement

Two's complement

Interpret the leading bit differently

Unsigned value: 200

$$2^7 + 2^6 + 2^3 = 200$$

Unsigned: 2^7

1	1	0	0	1	0	0	0
---	---	---	---	---	---	---	---

Two's complement: -2^7

Two's complement: -56

$$-2^7 + 2^6 + 2^3 = -56$$

Important properties

- Values can't exceed a range (overflow)
- Negation: $-y$ can be $\sim y + 1$

Binary (8-bit)	Unsigned [0, 255]	Signed (two's complement) [-128, 127]
00000000	0	0
00000001	1	1
00000010	2	2
00000011	3	3
...	...	...
01111101	125	125
01111110	126	126
01111111	127	127
10000000	128	-128
10000001	129	-127
10000010	130	-126
10000011	131	-125
...	...	...
11111101	253	-3
11111110	254	-2
11111111	255	-1